

React Router API Frontend

COMP50045

Michal Litwin

RICOH UK Products Ltd
Staffordshire University

Workplace Supervisor: Claire Shepherd

Software Engineering Route

1 Introduction

Development of React Router application through an Agile methodology. The web application communicates with a REST API built with Express.

1.1 Application Requirements

functional and non functional

1.2 Pre-Development

Initial weeks were used to meet module Tasks Tracker; including researching Git, Agile, Testing, Rendering Models and React. Evidence of research may be located in Appendices.

2 Development

2.1 Sprint 1: Application Design

Users must experience frictionless interactions incorporating accessibility needs and overall responsiveness. Including, keyboard only navigation, appropriate colours, screen readers, input assistance and compliance with WCAG standards.

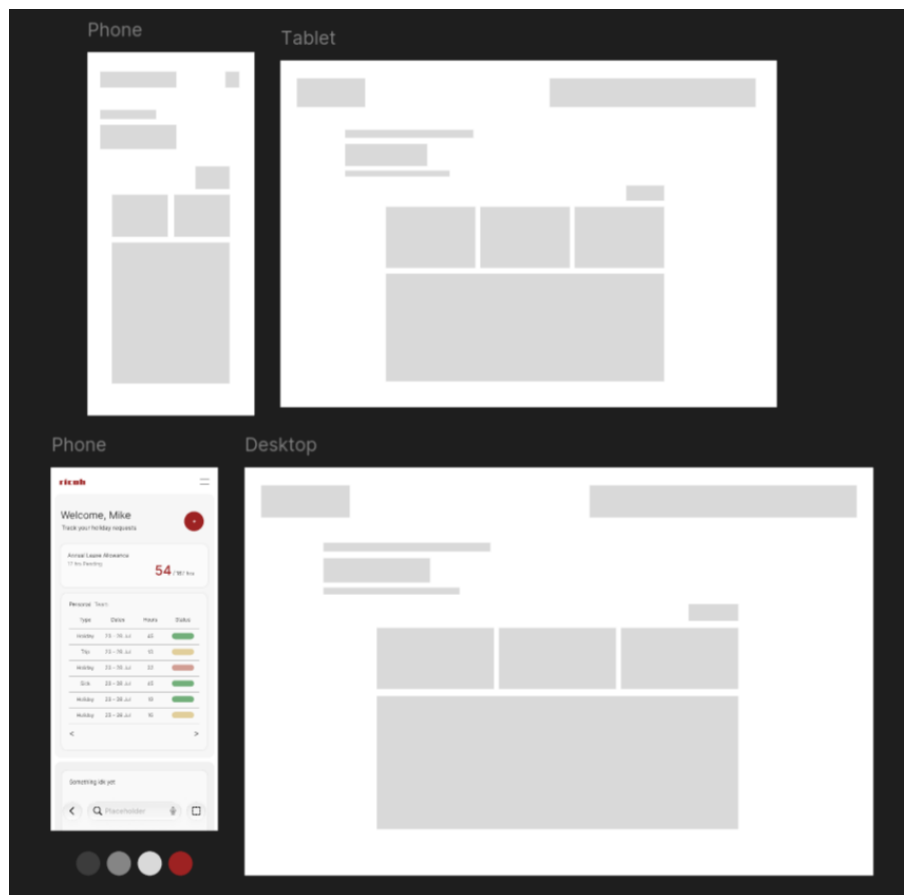


Figure 1: Application Wireframes & High Fidelity

As displayed in Figure 1 the application's main content is located in the centre of the visible space, with the logo and navigation on the top. Horizontal navigation on the top, provides an optimal and familiar location for users, as well as enables more space for components. High Fidelity diagram created to showcase potential final website interface, with colour, shadows, borders and placeholder text.

2.1.1 Agile

Designs were shared with group, displaying wireframes for different device formats as well as a high fidelity design. Feedback was limited due to the early stage of the overall development cycle. This sprint's duration exceeded initial time expectations, due to Analysis Paralysis (Appendices), concluding with a simple design to improve after the application meets original requirements.

2.2 Sprint 2: Backend Improvements

Postgres Database and API required updating to enable token refreshing, a logout option and overall improvements. New improvements improve user experience by storing refresh tokens, preventing the user from mandatory login after certain duration.

2.2.1 Database

Created 'refresh_token' and 'invalidated_jti' tables to provide reference for the API to determine user authorisation.

2.2.2 API

Developed a token API service providing CRUD functions for user refresh and invalidated access tokens table records. Enabled CORS to allow communication between the backend API and the React application. Login endpoint was updated to return the refresh token alongside the JWT, and the JWT expiration time was reduced to 2 minutes to strengthen security. This may be further improved by passing the refresh token directly into a http only cookie on login, instead of manually assigning a cookie in the React application. Additionally, cookie-based handling was enabled, and implemented at a new logout endpoint to check the cookie stored refresh token and delete the relevant associated 'refresh_token' record from the database.

2.2.3 Agile

2.3 Sprint X: Further

Add a log component in the admin to see errors and so on, need to modify api.

3 Notes

- refresh token tbl could also contain a refreshed_on column for more analytics
- refresh token rn is passed in jwt, this can be improved - send refresh token as cookie http only

Items to complete within this code design, creation, development plan, sprint conclusion, testing for each sprint

right now the access token will be manually set as a cookie in react, this can be improved in the api by also attaching the jwt to a cookie

3.1 Application Designs

- wireframes - hi/lo
- holiday page, requests page, admin page

3.2 Testing

3.3 Security

Access and refresh tokens used. Jwt on log out, the refresh token needs to be invalidated - this can be by adding a revoked_at column, or deleting the refresh token record entirely, forcing the user to log in and create one. the expires_at column could also be used to accommodate this, however this overloads the objective, and purpose as well as the identity of that column. For simplicity, user records are deleted on log out.

4 Appendices

4.1 User Experience/Interface

4.2 Analysis Paralysis

4.3 Git

4.4 Agile

4.5 Testing

4.6 Rendering Models

4.7 React

4.8 Tokens

4.9 Web Storage

cookies and jwt, vs local storage, data being in jwt, manipulation,